# Scalability: From 1 to N Projects

## Scalability: From 1 to N Projects I

The Standalone Project Paradigm enables horizontal scaling: a project supplies its manuscript/config, isolated source and tests, thin analysis scripts, and declared dependencies. New projects use existing infrastructure interfaces; admission to the tracked public roster and release matrix remains explicit. The `run.sh` orchestrator automatically discovers new projects and presents them in its interactive menu.

We have validated scaling with 25 canonical exemplars under `projects/templates/`—always present for onboarding and tooling—and with this manuscript from `projects/templates/template_template` (158 tests) as a git-tracked public exemplar in the same automated discovery menus.

# Scalability: From 1 to N Projects I

Canonical trio:

## Scalability: From 1 to N Projects I

- **`template_code_project`**: Numerical optimization example with gradient-descent narration and 258 discovered tests. Its `pyproject.toml` declares the 90% project-source floor. Minimal footprint: compact `src/`, scripted analysis, short manuscript sections.
- **`template_prose_project`**: Prose-heavy manuscript emphasizing narrative structure and bibliography discipline, with 165 discovered tests and the same declared project-source floor; tests exercise rendering and Markdown integrity without heavyweight numerics.
- **`template_autoresearch_project`**: AutoResearch readiness workflow — bounded ML task loop (deterministic MNIST run), plan validation (`infrastructure.autoresearch`), evidence-registry reports, and manuscript hydration; its own scripts (`run_autoresearch_loop.py`, `z_generate_manuscript_variables.py`) stay thin over `src/` logic. **`template_search_project`** exercises literature-search workflows (remote corpus JSON fusion, `y_generate_search_figures.py`, review reports).

## Scalability: From 1 to N Projects I

Meta manuscript (**`projects/templates/template_template`**) analyzes the repository via `src/template_template/` introspection metrics; it now lives alongside the other public exemplars under `projects/templates/`.

These workspaces share no project-level code—only Layer 1 (31 infrastructure subdirectories, ~1355 Python files)—validating insulation between domain repos and reusable services.

## Multi-Project Orchestration I

When the `--all-projects` flag is passed to `run.sh`, the pipeline executes each discovered project sequentially, running infrastructure tests once at the start and skipping them for individual projects to avoid redundant validation. After all projects complete, a cross-project executive report aggregates metrics (test counts, coverage percentages, page counts, rendering durations) into a unified dashboard with both JSON and Markdown output formats. This executive reporting stage provides repository-level visibility without requiring any project-specific reporting code.

# Scaling Metrics I

| Metric | template_code_project | template_prose_project | template_autoresearch_project |
|---|---|---|---|
| Source modules | 25 | 8 | 59 |
| Test files | 16 | 13 | 23 |
| Test count | 258 | 165 | 318 |
| Manuscript chapters | 9 | 8 | 6 |
| Analysis scripts | 9 | 5 | 5 |
| Figures (auto-generated) | 9 | 5 | 27 |

## Scaling Metrics I

The infrastructure overhead per project is constant regardless of project size: the same 31 modules, the same 17 declared pipeline stages, the same rendering and validation logic. The shared implementation does not grow with the project roster; execution time and output volume still grow with the work performed.